

AI Defence Lab

Adaptive Attack Family Design v1 · 24 August 2026

Purpose

The final loop should be a stable benchmark with evidence-driven adaptation. The approved taxonomy keeps experiments comparable; detector weaknesses guide the next family or variant; new patterns become official only after validation.

The family levels

Approved benchmark families: account_takeover, trusted_device, beneficiary_manipulation, low_and_slow, social_engineering, merchant_abuse, cross_channel_anomaly.

Adaptive variants: trusted_device_normal_velocity, low_and_slow_common_channel, beneficiary_manipulation_moderate_amount.

Discovery candidates: combinations such as trusted_device_low_and_slow. These remain candidates until they pass novelty, realism, generator, and repeated-seed checks.

How one round works

Base documents provide general payment-security knowledge. Attack Memory provides previous detector weaknesses. Agent 1 uses both to recommend a direction. The controller validates it, Agent 2 writes constraints, the generator creates rows, and the detector evaluates unseen rows.

Knowledge + memory → Agent 1 recommendation → controller validation → Agent 2 specification → generator → detector → Agent 3 weakness → memory

Simple example

Round 1: The seeded schedule starts with account_takeover. The detector misses attacks that use normal devices.

Round 2: Agent 1 recommends trusted_device with normal device behavior, moderate beneficiary changes, ordinary velocity, and controlled amounts.

Round 3: If the detector still misses attacks spread over time, Agent 1 recommends low_and_slow.

Round 4: If both weaknesses combine, Agent 1 proposes trusted_device_low_and_slow. The controller records it as a composite candidate, not an unreviewed new family.

How the recommendation is controlled

Agent 1 proposes	family, variant or candidate; reason; target weakness; confidence
Controller checks	approved taxonomy, relevance to weakness, recent repetition, generator support
Agent 2 writes	time, amount, device, beneficiary, channel and evasion constraints
Generator validates	schema, ranges, labels, family purity, diversity and reference similarity
Promotion gate	novelty, realism, generator support and repeated multi-seed evidence

How realistic quality is protected

The LLM does not generate raw transaction rows. It proposes intent and constraints. Family profiles and the learned generator create the data, while validators keep it within the synthetic schema.

- Family profiles map each direction to feature relationships.
- Agent 2 states what should remain normal and what should change.
- Amounts, hours, binary fields, velocity, channels and labels are validated.
- Generated attacks are compared with legitimate synthetic reference data.
- Mixed-family rows are rejected rather than silently relabelled.
- Unseen rows are kept separate from generator and detector training data.

Final policy

1. Start with a seeded approved family. 2. Use the latest weakness to guide Agent 1. 3. Validate the recommendation. 4. Let Agent 2 create structured constraints. 5. Generate and evaluate. 6. Store the weakness. 7. Promote candidates only after repeated evidence.

Stable benchmark + adaptive variants + controlled discovery

The taxonomy is versioned: taxonomy_v1, taxonomy_v2, and so on. This preserves old reports while allowing the method to improve. The result is neither a completely fixed schedule nor uncontrolled automatic discovery.

This design note describes the intended method. Metrics from the existing baseline and generator reports remain separate experiment evidence and are not official Mastercard scores.